ISTA 130
Midterm Review

In this PDF:

- Complex boolean statements
- Complex loops
- Function interactions
- Random numbers
- String Indexing

Boolean Expressions

IMPORTANT: A boolean expression is not the same as a conditional statement!

A boolean expression is any statement that will evaluate to either True or False:

What do the following boolean expressions evaluate to?

```
5 < 4  ->  ???

10 >= 10 and 10/4 == 2.5  ->  ???

'Hello' == 'Goodbye'  ->  ???

'1140'.isdigit()  ->  ???
```

Boolean Expressions

IMPORTANT: A boolean expression is not the same as a conditional statement!

A boolean expression is any statement that will evaluate to either True or False:

What do the following boolean expressions evaluate to?

```
5 < 4  ->  False

10 >= 10 and 10/4 == 2.5  ->  True

'Hello' == 'Goodbye'  ->  False

'1140'.isdigit()  ->  True
```

Boolean Expressions

The logical operators `and` and `or` combine two complete boolean expressions into one more complex expression. The `not` operator changes a boolean to its opposite

a =	z =		a and z	a or z	not a
True	True	---->	True	True	False
True	False	---->	False	True	False
False	True	---->	False	True	True
False	False	---->	False	False	True

Boolean Expressions

Python evaluates complex boolean expressions from left to right and sometimes takes shortcuts. Knowing this can save you from some trouble. For example, the following two expressions may look like they would have the same output, but in some cases, they work differently:

	`x != 0 and 12 % x == 0`	`12 % x == 0 and x != 0`
`x = 3`	`True`	`True`

`x = 0`	`False`	`Traceback (most recent call last):`
		`  File "<stdin>", line 1, in <module>`
		`ZeroDivisionError: integer division or modulo by zero`

Boolean Expressions

So why does `x != 0 and 12 % x == 0` work,
but not `12 % x == 0 and x != 0`?

When `x = 0`, python reaches the first expression, `x != 0`, evaluates it to be `False`, and stops. It doesn't matter what the second part is, because the `and` means that one `False` makes that entire expression `False`. Python saves time by skipping the second part in this case.

In the second case, python attempts to evaluate the first expression, but immediately encounters an error with `12 % 0`. Errors stop the flow of the entire program, so you can't salvage this case.

Question: what would be a similar situation with `or`?

Returning

```
def square(value):
    number = value ** 2
    return number
def main():
    number = 4
    square(number)
    print(number)
```

After main runs, what is the value of `number`? This choice of variable names can make it hard to tell just by looking at the code, since the same variable was used in both functions. Let's try again.

Returning

```
def square(value):
    answer = value ** 2
    return answer
def main():
    number = 4
    square(number)
    print(answer)
```

This code doesn't work because `answer` only exists within the `square` function. Whatever value was stored in that variable isn't directly accessible outside of the function.

Returning

```
def square(value):
    answer = value ** 2
    return answer
def main():
    number = 4
    my_val = square(number)
    print(my_val)
```

If we want to do something with the result of the function, we have to capture the output in some way. In this case, we use a different variable to store the return value.

Returning

```
def square(value):
    answer = value ** 2
    return answer
def main():
    number = 4
    print(square(number))
```

We can also simply use the return value as soon as it comes out. Be careful with this method, though, because after the value prints, we can't refer back to the value later on in the code.

Returning

We aren't restricted to using functions and return values inside main(). Complex functions often require the use of multiple smaller functions to reduce work:

```
def square(value):
    answer = value ** 2
    return answer
def add_squares(val1, val2):
    return square(val1) + square(val2)
def main():
    c_square = add_squares(3, 4)
    print("c squared = " + str(c_square))
```

Returning

There's no set limit to how many functions you can use together.

```
def square(value):
    answer = value ** 2
    return answer
def sqrt(value):
    return value ** (1/2)
def add_squares(val1, val2):
    return square(val1) + square(val2)
def pythagoras(side1, side2):
    c_squared = add_squares(side1, side2)
    return sqrt(c_squared)
```

Loops

One very common practice with loops is keeping a count of something. This does not require a complex process, but does require some planning ahead. For example, if we assume the `is_prime()` function already exists:

```
num_primes = 0
for i in range(100):
    if is_prime(i):
        print(i)
        num_primes += 1
print(num_primes)
```

This code will count every prime between 0 and 99.

Loops

```
num_primes = 0
for i in range(100):
    if is_prime(i):
        print(i)
        num_primes += 1
print(num_primes)
```

Notice that the `num_primes` variable was created before the loop started. If it had been created within the loop, it would have reset to 0 with each loop, and the count would have never been more than 1.

While What?

It can often be difficult to decide on a condition for a while loop. Especially when using user input, the flow of your entire loop can depend on the location of one line of code. Both of these methods will provide the same output:

``
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
``
	``
	while True:
	    word = input("Enter word: ")
	    if word == 'end':
	        break
	    print("Your word: " + word)
	``

If we want to use the input as a condition, we have to create the variable before entering the loop and then take input again within the loop.	If we only want to get input once, we move it to the beginning of the loop and end based on an `if` statement, rather than the `while` condition.

While What?

```
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
```

```
while True:
    word = input("Enter word: ")
    if word == 'end':
        break
    print("Your word: " + word)
```

These two loops behave almost exactly the same, with the only difference being where you enter the loop. We'll break down the process step by step to see what's happening.

While What?

```
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
```

```
while True:
    word = input("Enter word: ")
    if word == 'end':
        break
    print("Your word: " + word)
```

1. Take input from the user

While What?

```
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
```

```
while True:
    word = input("Enter word: ")
    if word == 'end':
        break
    print("Your word: " + word)
```

1. Take input from the user
2. Check if we need to exit the loop

While What?

```
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
```

```
while True:
    word = input("Enter word: ")
    if word == 'end':
        break
    print("Your word: " + word)
```

1. Take input from the user
2. Check if we need to exit the loop
3. Print out a message

While What?

```
word = input("Enter word: ")
while word != 'end':
    print("Your word: " + word)
    word = input("Enter word: ")
```

```
while True:
    word = input("Enter word: ")
    if word == 'end':
        break
    print("Your word: " + word)
```

1. Take input from the user
2. Check if we need to exit the loop
3. Print out a message
4. Take input again and repeat

Random Numbers

In order to use random numbers in python we need to import the random module:

```
import random
```

The random module has several functions that work in different ways to generate different kinds of random numbers. For this lab, we will use the `randint` function.

Random Numbers

Once we've imported random, we can use the `randint` function to generate random numbers:

```
number = random.randint(0, 10) # Random number between 0 and 10
```

The randint function is inclusive on both ends. So the start value and end values are possible results:

```
number = random.randint(1,1) # Always returns 1!
```

Random Numbers

We can use seed values to make python give us the same random numbers over and over again:

```
random.seed(42)
```

Note that the seed value is an integer.

String Indexing

Indexing a string uses an index (place in the string) to refer to a character in the string. What will the expressions below evaluate to?

“Snake” [2] → ??

Word = “rocket”

Word[0] → ??

Word[2*2] → ??

Word[1] + Word[-1] → ??

String Indexing

Indexing a string uses an index (place in the string) to refer to a character in the string. What will the expressions below evaluate to?

“Snake” [2] → “a”

Word = “rocket”

Word[0] → “r”

Word[2*2] → “e”

Word[1] + Word[-1] → “ot”

Practice

Download midterm_review.py from D2L

Write each function following the instructions given in the docstrings.

This is not graded, but it is important that you complete activities to prepare for the midterm.